Lab 4 Assignment

Autoencoders and Temporal Convolutions in Current Models

Overview

In Lab 4 you built an autoencoder, a 1D CNN and a TCN on small synthetic problems. In this assignment you take one model that is used in current research or in production (2024 to 2026), read its paper, and rebuild its central idea at a scale that trains on a free Colab GPU. You then test one of the paper's claims yourself and look for a case where the model breaks.

Expect this to take most of the week. Several topics need a few hours of GPU time, and the first version rarely works, so start early.

Rules for all topics

1. Write the core of the model yourself in PyTorch. Libraries are fine for data loading, optimisers, metrics and plotting, and for loading pretrained weights where a topic says so. Packages that already implement the core component (for example `mamba-ssm`, `sae-lens`, `neuralforecast`, or a paper's own training code) may only be used to check your results. You may read the authors' code but not copy it; list the files you read in your report.
2. Use the Lab 4 model named in your topic as a baseline.
3. Choose one quantitative claim from the main paper of your topic and test it at your scale. Quote the claim, describe your setup and say whether your result supports it. A result that does not support the claim is acceptable if you explain the difference.
4. Run one ablation of the design choice that the paper presents as most important. Change only that choice and keep the parameter count within 10 %.
5. Report your main results as mean ± standard deviation over three seeds.
6. Find a setting in which your model clearly fails and explain why. The missing P wave in Lab 4, Part D, is the kind of failure meant here.
7. Choose hyperparameters on a validation split. Use the test split once, at the end.

Topics

Choose one topic.

Topic 1. Selective state-space models (Mamba-3, ICLR 2026)

Read: [1–4] **Lab 4 baseline:** TCN

a. Implement a diagonal linear state-space layer. Compute its output once as a recurrence and once as a convolution evaluated with the FFT, and show that the two agree to within 10^{-4}.

b. Implement a selective layer in which Δ, B and C depend on the input, as in Mamba. A sequential scan written in PyTorch is enough; no custom CUDA kernels are needed.

c. On the lag task of Lab 4 (Experiment C.2) with $P = 400$, compare the TCN (dilations chosen so that its receptive field covers the lag), the linear layer and the selective layer at matched parameter count. Report test MSE, time per training step and peak memory.

d. Implement the Selective Copying task from the Mamba paper [2] and check whether the time-invariant models (TCN and linear layer) fail where the selective layer succeeds.

e. Ablation: Mamba-3 [4] drops the short causal convolution that earlier Mamba blocks [2, 3] place before the state-space layer. Add a causal convolution with kernel size 4 in front of your selective layer and measure the effect on (c) and (d).

Topic 2. Pretrained forecasting models (Chronos-2, TimesFM 2.5, Moirai 2.0)

Read: [5–7] **Lab 4 baseline:** TCN with a quantile output

a. Use data that the models cannot have seen during pretraining, for example hourly weather for El Alamein from January 2026 onwards from the Open-Meteo historical weather API [8]. Explain why your data cannot be part of their training sets.

b. Produce zero-shot forecasts with at least two of the three models at horizons of 24 h and 168 h, with 80 % prediction intervals.

c. Train the Lab 4 TCN on the same history with a quantile (pinball) loss, and add a seasonal-naive forecast as a second baseline.

d. Report MASE and the empirical coverage of the 80 % intervals, and measure inference time on a CPU.

e. Ablation: Chronos-2 [5] can use covariates through its group attention. Forecast temperature with and without one related variable (for example wind speed or humidity) and report the difference.

Topic 3. Representation autoencoders (RAE, 2025)

Read: [9–11] **Lab 4 baseline:** dense autoencoder of Section A.3, trained on CIFAR-10

a. Train a convolutional VAE on CIFAR-10 (32 × 32 images) with a latent of size 8 × 8 × 4, and report PSNR on the test set.

b. Build an RAE: a frozen DINOv2 ViT-S/14 encoder (available through `torch.hub`) followed by a decoder you design and train to reconstruct the image from the patch features. Report PSNR.

c. Train a linear classifier on the frozen VAE latents and on the frozen DINOv2 features, and compare CIFAR-10 accuracy. The RAE paper [10] reports a very large gap on ImageNet; check whether you find one.

d. Ablation: train decoders of three sizes. The RAE paper [10] reports that larger decoders reconstruct better.

e. Optional: train a small flow-matching or diffusion model in each latent space on one CIFAR-10 class and compare the samples.

Topic 4. Sparse autoencoders on a language model (Gemma Scope 2)

Read: [12–16] **Lab 4 baseline:** PCA (Section A.4) with as many components as the average number of active latents

a. Collect residual-stream activations from a middle layer of Gemma 3 270M [16] (or GPT-2 small) on at least 500,000 tokens of text.

b. Train a TopK sparse autoencoder (k = 32, at least eight times wider than the residual stream) and a ReLU + L1 sparse autoencoder tuned to the same average number of active latents. This comparison is your ablation.

c. For both, and for the PCA baseline, report the fraction of variance explained, the average number of active latents, the share of latents that never activate, and the increase in the model's next-token loss when the layer's activations are replaced by the reconstruction.

d. Interpret ten latents from their top-activating contexts. For two of them, write 20 sentences that should activate the latent and 20 that should not, and measure how well the latent separates them.

e. Optional, for Gemma 3 270M: compare your latents with the Gemma Scope 2 SAEs [15] for the same layer.

Topic 5. A streaming neural audio codec (SoundStream, Mimi)

Read: [17–20] **Lab 4 baseline:** convolutional autoencoder of Part D, adapted to audio, without quantisation

a. Build a causal encoder and decoder from strided convolutions (for example strides 2, 4, 5 and 8, i.e. 320 samples per frame at 16 kHz) and residual units with dilations 1, 3 and 9. Prove causality with the `is_causal` test from Lab 4 and state the algorithmic latency in milliseconds.

b. Write residual vector quantisation with at least four codebooks yourself, with quantiser dropout so that one model can run with 1, 2 or 4 codebooks. Report how evenly each codebook is used.

c. Train on at most 10 hours of LibriSpeech with a multi-scale spectrogram loss and an L1 loss on the waveform. Adversarial training is not required.

d. Report bitrate, SI-SDR and a mel-spectrogram distance at 1, 2 and 4 codebooks, and compare with the pretrained EnCodec [18] and Mimi [20] models from Hugging Face on the same clips.

e. Ablation: the same codec with non-causal (symmetric) padding. What does causality cost?

Topic 6. Large kernels instead of dilation (ModernTCN, ICLR 2024)

Read: [21–24] **Lab 4 baseline:** TCN

a. Use ETTh1 and ETTm1 [24] with the standard chronological splits, an input window of 336 steps and horizons of 96 and 336 steps. Report MSE and MAE on normalised data.

b. Implement a ModernTCN-style model following the paper [21]: a patch embedding, then blocks made of a large-kernel depthwise convolution followed by two pointwise feed-forward layers, one within each variable and one across variables.

c. Add a DLinear model [23] as a second baseline.

d. Measure the effective receptive field of the TCN and of your ModernTCN-style model with the autograd method of Lab 4, and plot input sensitivity against lag.

e. Ablation: at least three kernel sizes for the large depthwise convolution, at comparable parameter count. Does accuracy follow the effective receptive field?

What to submit

One zip file, `Lab4Assignment_<StudentID>.zip`, containing:

1. the notebook. It must run from top to bottom on a free Colab GPU in at most 30 minutes. If training takes longer, save checkpoints, include a download link, and add a flag `TRAIN = False` that loads them;
2. a report of at most four pages (PDF): the method in your own words (at most 300 words), the claim you tested and the outcome, results over three seeds, the ablation, the failure case, the list of the authors' files you read, and your declaration of AI use;
3. the slides for your presentation.

Presentation in Lab 5

Five minutes of presentation followed by five minutes of questions. Expect to:

- explain any line of your code;
- predict the effect of a change before it is run live;
- say which result would make you change your conclusion.

A part of your code that you cannot explain will not be accepted as your work.

Use of AI tools

You may use AI tools to understand the papers and to debug. Describe how you used them in the report. You must be able to explain everything you submit, however it was produced.

References

[1] A. Gu, K. Goel, C. Ré. Efficiently Modeling Long Sequences with Structured State Spaces. ICLR 2022. arXiv:2111.00396.
[2] A. Gu, T. Dao. Mamba: Linear-Time Sequence Modeling with Selective State Spaces. COLM 2024. arXiv:2312.00752.
[3] T. Dao, A. Gu. Transformers are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality (Mamba-2). ICML 2024. arXiv:2405.21060.
[4] A. Lahoti et al. Mamba-3: Improved Sequence Modeling using State Space Principles. ICLR 2026. arXiv:2603.15569.
[5] A. F. Ansari et al. Chronos-2: From Univariate to Universal Forecasting. arXiv:2510.15821, 2025.
[6] A. Das, W. Kong, R. Sen, Y. Zhou. A decoder-only foundation model for time-series forecasting (TimesFM). ICML 2024. arXiv:2310.10688. Version 2.5 (September 2025): github.com/google-research/timesfm.
[7] C. Liu et al. Moirai 2.0: When Less Is More for Time Series Forecasting. arXiv:2511.11698, 2025.
[8] Open-Meteo. Historical Weather API. open-meteo.com/en/docs/historical-weather-api.
[9] R. Rombach et al. High-Resolution Image Synthesis with Latent Diffusion Models. CVPR 2022. arXiv:2112.10752.
[10] B. Zheng, N. Ma, S. Tong, S. Xie. Diffusion Transformers with Representation Autoencoders. ICLR 2026. arXiv:2510.11690.
[11] M. Oquab et al. DINOv2: Learning Robust Visual Features without Supervision. TMLR 2024. arXiv:2304.07193.
[12] H. Cunningham et al. Sparse Autoencoders Find Highly Interpretable Features in Language Models. ICLR 2024. arXiv:2309.08600.
[13] L. Gao et al. Scaling and evaluating sparse autoencoders. ICLR 2025. arXiv:2406.04093.
[14] A. Karvonen et al. SAEBench: A Comprehensive Benchmark for Sparse Autoencoders in Language Model Interpretability. ICML 2025. arXiv:2503.09532.
[15] T. Lieberum et al. Gemma Scope: Open Sparse Autoencoders Everywhere All At Once on Gemma 2. BlackboxNLP 2024. arXiv:2408.05147. Gemma Scope 2 for Gemma 3 270M: huggingface.co/google/gemma-scope-2-270m-pt.
[16] Gemma Team. Gemma 3 Technical Report. arXiv:2503.19786, 2025.
[17] N. Zeghidour et al. SoundStream: An End-to-End Neural Audio Codec. IEEE/ACM Transactions on Audio, Speech, and Language Processing, 2022. arXiv:2107.03312.
[18] A. Défossez et al. High Fidelity Neural Audio Compression (EnCodec). TMLR 2023. arXiv:2210.13438.
[19] R. Kumar et al. High-Fidelity Audio Compression with Improved RVQGAN (DAC). NeurIPS 2023. arXiv:2306.06546.
[20] A. Défossez et al. Moshi: a speech-text foundation model for real-time dialogue. arXiv:2410.00037, 2024.
[21] D. Luo, X. Wang. ModernTCN: A Modern Pure Convolution Structure for General Time Series Analysis. ICLR 2024.
[22] S. Bai, J. Z. Kolter, V. Koltun. An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling. arXiv:1803.01271, 2018.
[23] A. Zeng et al. Are Transformers Effective for Time Series Forecasting? (DLinear). AAAI 2023. arXiv:2205.13504.
[24] H. Zhou et al. Informer: Beyond Efficient Transformer for Long Sequence Time-Series Forecasting. AAAI 2021. arXiv:2012.07436.